SECURITY AUDIT REPORT

Elyra Android app

xyz.getelyra.app - Expo / React Native - Solana dApp Store

Scope: mobile/ (source, configuration, build, dependencies) Date: 2 November 2026 Version 1.0

At a glance

Findings	High	Medium	Low	Info	Fixed in this audit
11	**1**	**2**	**5**	**3**	**6** (+1 partly)

Overall the app is **well built from a security point of view**: one server, tokens never persisted, a strict permission set, a hardened WebView, telemetry scrubbed of PII, and push notifications bound to the signed-in account. The audit found one serious bug: a **deep-link allowlist bypass in release builds**. It also found a **design gap in transaction signing**: embedded-wallet signatures were made without the bytes ever being checked against what the confirm screen showed. Both are fixed in the working tree with tests. The main item left for you is operational: **a Firebase Admin private key and the release keystore sit in the project folder**.

Checks after the fixes: typecheck clean, lint clean, Jest 236 of 236 passing (221 before; 15 tests added). Nothing has been committed.

1. Executive summary

This report covers a security review of the Elyra Android app (`mobile/`), an Expo / React Native app that is published on the Solana dApp Store and shares its backend, accounts and wallets with getelyra.xyz. The review covered every source file in `mobile/src` (202 files), the native and build configuration, the dependency tree, and the repository history for leaked secrets.

What needed fixing

- **ELY-H-01 (High, fixed).** Any link containing the text `expo-development-client` skipped the deep-link allowlist, in production builds too. So a link on a web page or in a QR code could open any screen with any parameters, for example a swap pre-filled with a scam token.
- **ELY-M-01 (Medium, fixed).** Privy's Expo SDK signs with no confirmation screen, so the app's confirm step is the only one the person sees, and that step showed the server's description of the transaction rather than the transaction itself. A new on-device decoder now checks transfers exactly (recipient, token, amount, signer, fee, allowed instructions), and checks every signature for the right signer, before anything is signed.
- **ELY-M-02 (Medium, your action).** The Firebase Admin SDK private key and the Android keystore sit in `mobile/`. They are gitignored and were never committed or uploaded, but they belong in a secrets manager or offline storage, not a Desktop project folder.
- **Low and info.** External links opened with any scheme (fixed), the signed PDF URL was not checked for HTTPS (fixed), MWA signature prefix was not checked (fixed), dependency advisories in build tools and zod (open), swaps are only partly verified (partly fixed), and screenshots are not blocked (suggestion).

Overall rating

Before this audit: **Medium risk**, driven by the deep-link bypass combined with headless signing. After the fixes: **Low risk**, with ELY-M-02 to handle outside the code and the swap check (ELY-L-05) as the main structural follow-up. The rest of the codebase shows deliberate security thinking. Section 5 lists the controls that are already good.

2. Scope and method

In scope

- `mobile/src/**`: 37 screens, 92 components and 73 library modules, read in full where security-relevant (auth, session, API client, deep links, notifications, wallet and signing, WebView, external links, storage, crash reporting, attachments, account deletion).
- `app.config.ts`, `eas.json`, `metro.config.js`, `entrypoint.js`, `.env`/`.env.example`, `.gitignore`, and the generated `AndroidManifest.xml` and Gradle files.
- The shared server code the app depends on, read only to confirm contracts: `lib/solana/transfer.ts`, `lib/solana/fees.ts`, `lib/trade.ts` (toBaseUnits), `lib/markdown.ts`, `lib/notifications/routes.ts` and `app/api/transfer/build`.
- Dependencies: `pnpm audit --prod` over 961 packages. Secrets: a pattern scan of every tracked file in `mobile/` plus file names across all git history.

Method

- Manual source review against OWASP MASVS v2 and the threats specific to a crypto wallet: getting the person to sign something they did not intend, deep-link phishing, a compromised backend, a malicious wallet app, and data left on a shared phone.
- Every claim was checked rather than assumed. The Privy SDK was inspected for confirmation UI, the server's transaction construction was read, and the deep-link bypass was reproduced with a release-mode probe.
- Each fix comes with tests. The transaction fixtures were generated with `@solana/kit` in exactly the way the server builds transactions, so the checks run against real bytes.

Not in scope / not done

- **The server** (`app/api`, most of `lib/`) and the website. You asked for these to come later. Several recommendations here (ELY-L-05) belong on the server.
- Dynamic testing on an emulator or device, traffic interception, and reverse engineering of a built APK. CLAUDE.md requires a device check for signing and MWA, and that should be done before release.
- Third-party services (Privy, Jupiter, Sentry, Firebase) beyond how the app uses them.

Severity scale

Severity	Meaning
High	Directly exploitable by an outside party with little interaction, and it weakens a stated security boundary.
Medium	Exploitable under realistic but specific conditions (backend compromise, a specific social-engineering set-up, or local access to the developer's machine).
Low	Defence in depth, or limited impact.
Info	Hardening or hygiene suggestion; no direct vulnerability.

3. Findings summary

ID	Finding	Severity	Status
ELY-H-01	Deep-link allowlist bypassed in release builds	**High**	**Fixed**
ELY-M-01	Signing server-built bytes that were never checked against the confirm screen	**Medium**	**Fixed**
ELY-M-02	Firebase Admin SDK private key and the release keystore sit in the project folder	**Medium**	**Action needed**
ELY-L-01	External links opened with any URL scheme	**Low**	**Fixed**
ELY-L-02	Signed report-PDF URL downloaded without a scheme check	**Low**	**Fixed**
ELY-L-03	Mobile Wallet Adapter signature parsing accepted any prefix	**Low**	**Fixed**
ELY-L-04	Vulnerable packages in the dependency tree (build tooling, plus zod)	**Low**	**Open**
ELY-L-05	Swap transactions are not checked instruction by instruction	**Low**	**Partly fixed**
ELY-I-01	No screenshot or app-switcher protection on financial screens	**Info**	**Open**
ELY-I-02	Minor hardening	**Info**	**Fixed**
ELY-I-03	Build posture notes	**Info**	**Accepted**

4. Detailed findings

ELY-H-01 Deep-link allowlist bypassed in release builds

Severity: **High** Status: **Fixed** MASVS: MASVS-PLATFORM-1

Location: `mobile/src/lib/links.ts:58 (before the fix), called from src/app/+native-intent.tsx`

Description

Every link that opens the app goes through `resolveLink`, which is meant to be a strict allowlist. Its first line, though, handed any link straight to the router if the string **contained** the text `expo-development-client` anywhere, or started with `exp://`/`exp+elyra://`. The exception was not limited to development builds, so it applied in production too.

Impact and attack scenario

Anyone who can get the person to tap a link (a web page, a QR code, a chat message, a notification from another app) can open **any** screen in the app with **any** parameters, skipping every check in the allowlist. Examples:

- `elyra://swap?from=SOL&to=<scam token>&amount=50&x=expo-development-client` opens the swap screen pre-filled with an attacker-chosen pair and amount, a classic phishing set-up.
- `elyra://delete-account#expo-development-client` lands the person on the account-deletion flow.
- `elyra://transfer?intent=withdraw&...` and other internal routes open the same way.

Every money movement still needs the person to review and confirm, which is why this is not rated Critical. But it removes the boundary the code says it has, and a pre-filled screen that looks legitimate is exactly what social-engineering attacks depend on.

Evidence

Reproduced with a Jest probe calling `resolveLink(raw, { development: false })`, the same call a release build makes. All four crafted links came back `{"kind":"passthrough"}`:

```
elyra://swap?from=SOL&to=BONK&amount=50&x=expo-development-client  => passthrough
elyra://delete-account#expo-development-client                    => passthrough
exp+elyra://swap?from=SOL                                         => passthrough
exp://evil.example/--/swap                                        => passthrough
```

Fix applied

Dev-client links now pass through **only** when `development` is true (`__DEV__`), and only when the link **starts** with the dev-client form (`DEVELOPMENT_CLIENT` regex, links.ts:31). In a release build every link is checked against the allowlist. The same four links now resolve to `ignored` (the home screen). New tests cover both build types.

ELY-M-01 Signing server-built bytes that were never checked against the confirm screen

Severity: **Medium** Status: **Fixed** MASVS: MASVS-CODE-4 / MASVS-PLATFORM-3

Location: `mobile/src/lib/wallet/signer.ts, src/app/transfer.tsx, src/app/swap.tsx`

Description

The server builds every transaction and the app signs it, which is a sound design. But two facts combined into a gap:

- **The embedded wallet has no confirmation screen of its own.** The installed `@privy-io/expo` 0.77.0 signs headlessly. There is no confirmation UI in the SDK, and `swap.tsx`'s own comment says "the embedded wallet has no screen of its own". A comment in `signer.ts` claimed the opposite ("Privy shows the embedded wallet's" confirmation).
- **The app's confirm screen shows the server's own description** (`summary`, `to`) and then signs the server's bytes. Nothing tied the two together, so they could disagree without anyone noticing.

Impact and attack scenario

If the API were compromised, misconfigured, or had a bug in transaction construction, the person could see "Send 1.5 SOL to 9WzD..." and sign a transaction that sends a different amount to a different address, or adds an extra instruction. For embedded-wallet users nothing else stands between the confirm button and the signature. It is rated Medium rather than High because exploiting it needs control of the API.

Evidence

Searched the SDK distribution for any confirmation strings or UI options: none. The signer passed `unsignedBase64` straight to `provider.request({ method: "signTransaction" })`.

Fix applied

New pure module `src/lib/wallet/inspect.ts` (280 lines, 11 tests), which decodes Solana wire transactions (legacy and v0, including address lookup tables):

- **Every signature** (embedded and Mobile Wallet Adapter): `requireSigner` refuses bytes that are unreadable or do not name the signing wallet as a required signer. This happens before either wallet sees the bytes (signer.ts:86, :118).
- **Transfers**: `checkTransfer` runs before the confirm screen is shown (transfer.tsx:183). It requires the exact shape `lib/solana/transfer.ts` builds: compute budget, then either a System transfer or a create-ATA-idempotent plus `transfer_checked`. The sender must be the only signer and fee payer. The recipient, mint and amount must equal what the person typed, using the same truncating arithmetic as the server's `toBaseUnits`. The priority fee is capped at 1,000,000 lamports (the server's maximum is 60,000). Any other program or instruction is refused.
- The token recipient is bound **on-chain**: the ATA program rejects an account not derived from its `owner` and `mint`, and `checkTransfer` requires `transfer_checked` to pay into exactly that account.
- The server's echoed `to` must equal the address typed. The misleading comment in `signer.ts` has been corrected.
- Fixtures are real transactions built with `@solana/kit` in the same way the server builds them. Tampered copies are refused: wrong recipient, redirected destination, changed amount, changed token, extra signer, Assign in place of Transfer, a foreign program, or a fee of about 4,295 SOL.

Residual risk

Swaps are only partly covered. Jupiter routes are not something the client can verify instruction by instruction, so swaps get the signer check only. See ELY-L-05.

ELY-M-02 Firebase Admin SDK private key and the release keystore sit in the project folder

Severity: **Medium** Status: **Action needed** MASVS: MASVS-STORAGE (developer environment)

Location: `mobile/elyra-mobile-firebase-adminsdk-fbsvc-7635920aae.json`, `mobile/@kunalempire__elyra.jks`

Description

The `mobile/` working directory contains a Firebase **service-account private key** for project `elyra-mobile` (type `service_account`, `private_key` present) and a Java KeyStore that appears to be the app's upload/signing key.

Impact and attack scenario

Good news first: both files are covered by `.gitignore` (root and mobile). Neither appears anywhere in git history (checked every commit on every branch), and EAS honours `.gitignore` when there is no `.easignore`, so neither is uploaded with a build. Neither is referenced by the app or the Metro config, so neither can end up in the bundle.

The risk is operational. An Admin SDK key gives full administrative access to the Firebase project, including sending push notifications to every user. A leaked signing key on the Solana dApp Store is especially serious, because the package's signing identity cannot be changed after the first release. Files in a Desktop project folder get read by IDE extensions, AI coding tools and backup software, and on macOS they are often synced to iCloud.

Recommendation

- Move both files out of the repository into a password manager or offline encrypted storage. Keep the keystore backup somewhere you will not lose it.
- If the FCM v1 key has already been uploaded to EAS (`eas credentials`), the local copy is no longer needed. If the folder has ever been synced or shared, rotate the service-account key in Google Cloud IAM.
- Add a `.easignore` (or keep relying on `.gitignore`) and consider a pre-commit secret scanner such as gitleaks.

I did not delete or move these files: they are yours and the action cannot be undone.

ELY-L-01 External links opened with any URL scheme

Severity: **Low** Status: **Fixed** MASVS: MASVS-PLATFORM-1

Location: `mobile/src/components/wallet/parts.tsx:195 (openExternal) and about 30 call sites`

Description

`openExternal(url)` passed its argument straight to `Linking.openURL`. Many of its URLs come from **third-party data**: report sources, Polymarket market URLs, token-unlock source URLs, SEC filing links, activity rows, and anything the TradingView widget navigates to.

Impact and attack scenario

A poisoned upstream record could carry a `solana:<address>?amount=...` payment request (opening the phone's wallet with a pre-filled transfer), an `intent:` URI, `tel:`, or another app's custom scheme in place of a web page. Agent replies were already safe: the shared Markdown parser only lets through `http(s)` and `mailto` links.

Fix applied

New pure `webUrl()` in `links.ts`:51 accepts only `http(s)://host...` with no whitespace, backslashes or userinfo. `openExternal` ignores anything else. There are 11 new test cases (`solana:`, `intent:`, `elyra:`, `tel:`, `javascript:`, userinfo, and empty host).

ELY-L-02 Signed report-PDF URL downloaded without a scheme check

Severity: **Low** Status: **Fixed** MASVS: MASVS-NETWORK-1

Location: `mobile/src/lib/pdf.ts:61 (reportPdf)`

Description

The API returns a short-lived signed URL, and the app downloaded it with `File.downloadFileAsync` without checking it was HTTPS.

Impact and attack scenario

This is defence in depth. A malformed or downgraded URL from the API could fetch over cleartext. Android also blocks cleartext in release builds, so the practical risk is low.

Fix applied

The function now refuses anything that is not an `https://` URL accepted by `webUrl`, and says so in the screen's usual error wording.

ELY-L-03 Mobile Wallet Adapter signature parsing accepted any prefix

Severity: **Low** Status: **Fixed** MASVS: MASVS-CRYPTO-2

Location: `mobile/src/lib/wallet/mwa.ts:92 (signatureFrom)`

Description

Wallets return either a bare 64-byte signature or the message with the signature appended. The app took the last 64 bytes whenever the length was `message.length + 64`, without checking that the first part really was the message it sent.

Impact and attack scenario

A misbehaving wallet could return a signature over different bytes, which the app would forward for Sign-In with Solana or a message signature. The server (Privy) verifies signatures, so the impact is limited.

Fix applied

The appended form is now accepted only if its prefix is byte-for-byte identical to the message that was sent. A test has been added.

ELY-L-04 Vulnerable packages in the dependency tree (build tooling, plus zod)

Severity: **Low** Status: **Open** MASVS: MASVS-CODE-3

Location: `mobile/pnpm-lock.yaml`

Description

`pnpm audit --prod` reports 18 advisories across 961 dependencies: 7 high, 7 moderate, 4 low, 0 critical. Eleven come from `node-forge 1.4.0` (through `@expo/cli`), and the rest from `glob`/`minimatch` (through Sentry's bundler plugin), `deepmerge` (through `@expo/config`) and `zod 3.25.76`.

Impact and attack scenario

All except zod are **build-time tools** and are not in the shipped Hermes bundle. Zod's advisory (GHSA-mrhx-c7xc-m3w5, memory exhaustion on oversized constrained collections) is fixed only in v4. In the app, zod is reached only through `lib/tracker/types.ts` on data from your own API, so it is not reachable by an outside attacker.

Recommendation

- Add `"pnpm": { "overrides": { "node-forge": "^1.4.2" } }` to mobile/package.json and reinstall. The other build-tool advisories will clear as Expo and Sentry update.
- Plan the zod v4 migration together with the web app, which shares `lib/`.
- Run `pnpm audit --prod` in CI and fail the build on new runtime advisories.

Not applied here: it rewrites the lockfile, which already has uncommitted changes of yours, and it does not change the shipped app.

ELY-L-05 Swap transactions are not checked instruction by instruction

Severity: **Low** Status: **Partly fixed** MASVS: MASVS-CODE-4

Location: `mobile/src/app/swap.tsx (sign)`

Description

A Jupiter swap routes through programs and lookup tables the client cannot enumerate. Ultra orders can also be gasless, with a fee payer other than the user. So a strict allowlist like the one transfers now use would break legitimate swaps.

Impact and attack scenario

A compromised API could still get an embedded-wallet user to sign a swap that does not match the quote on screen. Only the signer check (ELY-M-01) now applies.

Recommendation

- On the server, in `/api/swap/build`, decode Jupiter's transaction before returning it. Check that the taker is the user, that the user's token-account changes match the quote, and that no Token `SetAuthority`/`Approve` or System `Assign` instruction is aimed at the user's accounts.
- Optionally, on the client, refuse any top-level `SetAuthority`, `Approve` or `CloseAccount` aimed at the user's own token accounts (this is cheap with `inspect.ts`).
- Verify a real swap on a device after this change. CLAUDE.md requires device testing for signing, and the decoder has been tested against real kit-built v0 transactions but not yet against a live Jupiter order.

ELY-I-01 No screenshot or app-switcher protection on financial screens

Severity: **Info** Status: **Open** MASVS: MASVS-PLATFORM-3

Location: `whole app (no FLAG_SECURE, no expo-screen-capture)`

Description

Balances, positions, wallet addresses and exchange equity show in the Recents thumbnail, and can be captured by screenshots or screen-recording apps.

Impact and attack scenario

This is a privacy issue, not a key-loss risk. The app never shows a private key (export happens on the website), and a 'hide balances' toggle already exists.

Recommendation

- Use `expo-screen-capture` (`preventScreenCaptureAsync`) on the wallet, portfolio and connection screens, or blur the app when it goes to the background.

ELY-I-02 Minor hardening

Severity: **Info** Status: **Fixed** MASVS: -

Location: `links.ts, signer.ts`

Description

(a) `elyra://asset/..` matched the symbol pattern. Symbols must now start with a letter or digit. (b) The comment in `signer.ts` about Privy showing a confirmation was wrong, and wrong security comments mislead future reviewers. It now explains that the calling screen is the only confirmation.

Fix applied

Both changed, with tests.

ELY-I-03 Build posture notes

Severity: **Info** Status: **Accepted** MASVS: MASVS-RESILIENCE

Location: `app.config.ts, mobile/android/ (local prebuild)`

Details

- R8 minification is off on purpose, because it can strip classes that Privy, MWA and Sentry load by reflection. The JavaScript is Hermes bytecode either way. Acceptable, given there are no secrets in the bundle.
- The local `android/` folder is a stale prebuild: its manifest still lists `USE_BIOMETRIC`/`FOREGROUND_SERVICE`, which `app.config.ts` now blocks, and its release build type signs with `debug.keystore`. EAS regenerates it, so store builds are unaffected. Delete it or run `npx expo prebuild --clean` so nobody ships a local release by mistake.
- There is no certificate pinning. Android 7+ does not trust user-installed CAs for this app, and cleartext is blocked in release. Pinning is optional, and Privy and the API's CDN make it costly to maintain.
- The local `.env` points `EXPO_PUBLIC_API_URL` at an `http://` development server. Make sure the EAS `production` environment sets the `https://api.getelyra.xyz` URL (a cleartext URL would fail closed in release).

5. Controls already in place

These are working as intended and should be kept. Several are better than typical for wallet apps.

Area	What is in place
Session and tokens	The Privy session is kept in the device's secure storage. Access tokens are never persisted by the app, and the Zustand persist list excludes them along with conversations and attachments. A session-version guard drops a token fetched for an account that has since left.
One server only	The app talks only to EXPO_PUBLIC_API_URL. Every authenticated request carries a bearer token, and the server derives the account from that token. A 401 signs the person out once, with a reason.
Secrets hygiene	Only public EXPO_PUBLIC_ values are in the bundle. No secret has ever been committed (full history scan). The only token-like string in tracked files is a fake JWT in a test.
Platform hardening	allowBackup is false. The permission list is minimal, and a long block-list removes library-added permissions. Each permission is requested in context. Notification lockscreen visibility is PRIVATE.
WebView boundary	The TradingView chart has an origin allowlist plus onShouldStartLoadWithRequest, no file access, mixedContentMode='never' and no multiple windows. Config JSON is escaped against breaking out of the script tag. The only message bridge is 'ready'/'error'.
Rendering model output	Agent Markdown is rendered natively with no HTML path. Links are limited to http(s)/mailto, and reply sources come from tool results rather than model text.
Push notifications	A tap is acted on only after the payload's account is matched against the signed-in account or the authenticated inbox. Route data goes through an allowlist (lib/notifications/routes.ts).
Signer selection	Only the wallet holding the address the server built for may sign. A wallet app connected with a different key is refused before it sees any bytes. Signing never submits.
Transfer UX	The recipient is shown in full and can be selected. The person's own address is refused. A scanned Solana Pay QR fills only the address, never the amount or memo. A stale quote (over 20 s) is re-quoted before signing.
Exchange keys	Binance keys are entered in a masked field with autofill and autocorrect off, cleared from state immediately on submit, and checked as read-only by the server.
Crash reporting	Off in development. sendDefaultPii is off; there are no screenshots, view tree or request bodies. A thorough scrubber removes JWTs, bearer tokens, push tokens, Privy IDs, emails, addresses, signatures and query strings. Console breadcrumbs are dropped.
Data lifecycle	Signing out clears cached PDFs, the MWA authorization and the conversation. Account deletion wipes every secure-store key, AsyncStorage and the cache. Avatars are re-encoded, which strips EXIF/GPS.
Route parameters	Every dynamic route parameter that goes into an API path is passed through encodeURIComponent.

6. Dependency audit

`pnpm audit --prod` was run on 2 November 2026 in `mobile/`. It reported 18 advisories: 7 high, 7 moderate, 4 low, 0 critical.

Package	Path	Advisories	Exposure	Fixed in
node-forge 1.4.0	via @expo/cli	11 (5 high)	Build tool	>= 1.4.1 (override)
zod 3.25.76	direct	1 high	Runtime (reachable only via your API)	>= 4.6.6 (major)
glob 10.5.0 / 13.0.6	via @sentry/bundler-plugins	1 moderate	Build tool	>= 14.0.2
minimatch 9.0.9 / 10.2.6	via glob	1 moderate	Build tool	>= 11.0.1
deepmerge 4.3.1	via @expo/config	1 low	Build tool	none yet

Suggested override, to add to mobile/package.json and then run `pnpm install`:

```
"pnpm": {
  "overrides": {
    "node-forge": "^1.4.2"
  }
}
```

7. OWASP MASVS v2 mapping

Control group	Result	Notes
MASVS-STORAGE	Pass (app); developer workstation note (ELY-M-02)	Secure store for session, MWA token and referral; no backup; full wipe on deletion
MASVS-CRYPTO	Pass	No in-app cryptographic secrets; signatures by Privy or the wallet app; prefix check added (ELY-L-03)
MASVS-AUTH	Pass	Privy (email OTP, Google, passkey, SIWS); server derives identity from the bearer token; 401 sign-out
MASVS-NETWORK	Pass	HTTPS-only release; system trust store; no user CAs; no pinning (optional)
MASVS-PLATFORM	Fixed	Deep-link bypass (ELY-H-01) and external-scheme opening (ELY-L-01) fixed; WebView hardened
MASVS-CODE	Partly fixed	Transaction verification added (ELY-M-01); dependency advisories open (ELY-L-04); swap check partial (ELY-L-05)
MASVS-RESILIENCE	Accepted	No obfuscation (deliberate); not a goal for this app
MASVS-PRIVACY	Pass, one suggestion	Minimal permissions, scrubbed telemetry; screenshot protection suggested (ELY-I-01)

8. What to do next

When	Action	Owner
Now	Review and commit the fixes in this report (section 9). Then test on a device: a SOL transfer, an SPL transfer (including a Token-2022 mint), a max send, a swap through the embedded wallet, and a swap through MWA (Phantom or Seed Vault).	You
Now	Move the Firebase Admin key and the keystore out of mobile/ (ELY-M-02). Rotate the service-account key if the folder was ever synced or shared.	You
This week	Add the node-forge override. Add pnpm audit and a secret scanner to CI. Confirm the EAS production environment uses the https API URL.	You
Next release	Server-side check of Jupiter swap transactions in /api/swap/build (ELY-L-05). Screenshot/app-switcher protection on financial screens (ELY-I-01). Delete the stale android/ prebuild.	Server + app
Later	Audit the server (app/api, lib/) and the website, as planned. Migrate to zod v4 across web and app.	Next audit

9. Appendix: changes made

All changes are in the working tree under `mobile/` and are **uncommitted**, alongside your existing uncommitted work. Nothing outside `mobile/src` was changed.

File (mobile/)	Change	Purpose
`src/lib/wallet/inspect.ts`	new	Solana wire-transaction decoder; requireSigner; checkTransfer; toBaseUnits
`src/lib/wallet/inspect.test.ts`	new	11 tests built on real @solana/kit transactions, including tampered copies and lookup tables
`src/lib/wallet/signer.ts`	modified	requireSigner before every signature; corrected confirmation comment
`src/lib/wallet/signer.test.tsx`	modified	Real 32-byte addresses and well-formed bytes; new test that bytes for the wrong wallet are refused
`src/app/transfer.tsx`	modified	Recipient echo check and checkTransfer before the confirm screen
`src/lib/links.ts`	modified	Dev-client passthrough only in development and anchored; webUrl(); stricter SYMBOL
`src/lib/links.test.ts`	modified	Release-build bypass cases, webUrl cases, symbol case
`src/components/wallet/parts.tsx`	modified	openExternal opens only http(s)
`src/lib/pdf.ts`	modified	Signed PDF URL must be https
`src/lib/wallet/mwa.ts / mwa.test.ts`	modified	signatureFrom checks the message prefix; test

Checks

```
$ pnpm typecheck      tsc --noEmit                    clean
$ pnpm lint           expo lint                       clean
$ pnpm test           Test Suites: 34 passed, 34 total
                      Tests:       236 passed, 236 total   (221 before)
```

Key code: the transfer check

```
// src/app/transfer.tsx, review(): before the confirm screen is shown
if (payload.to !== trimmed) throw new TransactionCheckError("...different recipient...");
checkTransfer(payload.unsignedTransaction, {
  signer: payload.signer, recipient: trimmed, mint,
  amount: sendMax ? null : amount.trim(),
});

// src/lib/wallet/signer.ts: before either wallet sees the bytes
requireSigner(unsignedBase64, wallet.address);
```

Key code: the deep-link fix

```
// before (applied in release builds too, matched anywhere in the link)
if (/expo-development-client|^exp(\+[a-z-]+)?:\/\//i.test(raw)) return { kind: "passthrough" };

// after
if (development && DEVELOPMENT_CLIENT.test(raw.trim())) return { kind: "passthrough" };
```